

Advanced Algorithms

Tobias Achterberg



GUROBI
OPTIMIZATION

The World's Fastest Solver

What's Inside Gurobi Optimizer

- Algorithms for continuous optimization
- Algorithms for discrete optimization
- Automatic presolve for both LP and MIP
- Algorithms to analyze infeasible models
- Algorithms to solve multiple scenario MIPs
- Automatic parameter tuning tool
- Parallel and distributed parallel support
- Gurobi Compute Server and Cluster Manager
- Gurobi Instant Cloud
- Programming interfaces
- Gurobi modeling language based on Python
- Full-featured interactive shell

LP Algorithms

Primal and Dual LP

- Primal Linear Program:

$$\begin{array}{ll}\min & c^T x \\ \text{s.t.} & Ax = b \\ & x \geq 0\end{array}$$

- Weighted combination of constraints (**y**) and bounds (**z**):

$$y^T Ax + z^T x \geq y^T b \quad (\text{with } z \geq 0)$$

- Dual Linear Program:

$$\begin{array}{ll}\max & y^T b \\ \text{s.t.} & y^T A + z^T = c^T \\ & z \geq 0\end{array}$$

Obvious: Weak Duality

$$c^T x^* \geq y^{*T} b$$

(if primal and dual are both feasible)

Strong Duality Theorem:

$$c^T x^* = y^{*T} b$$

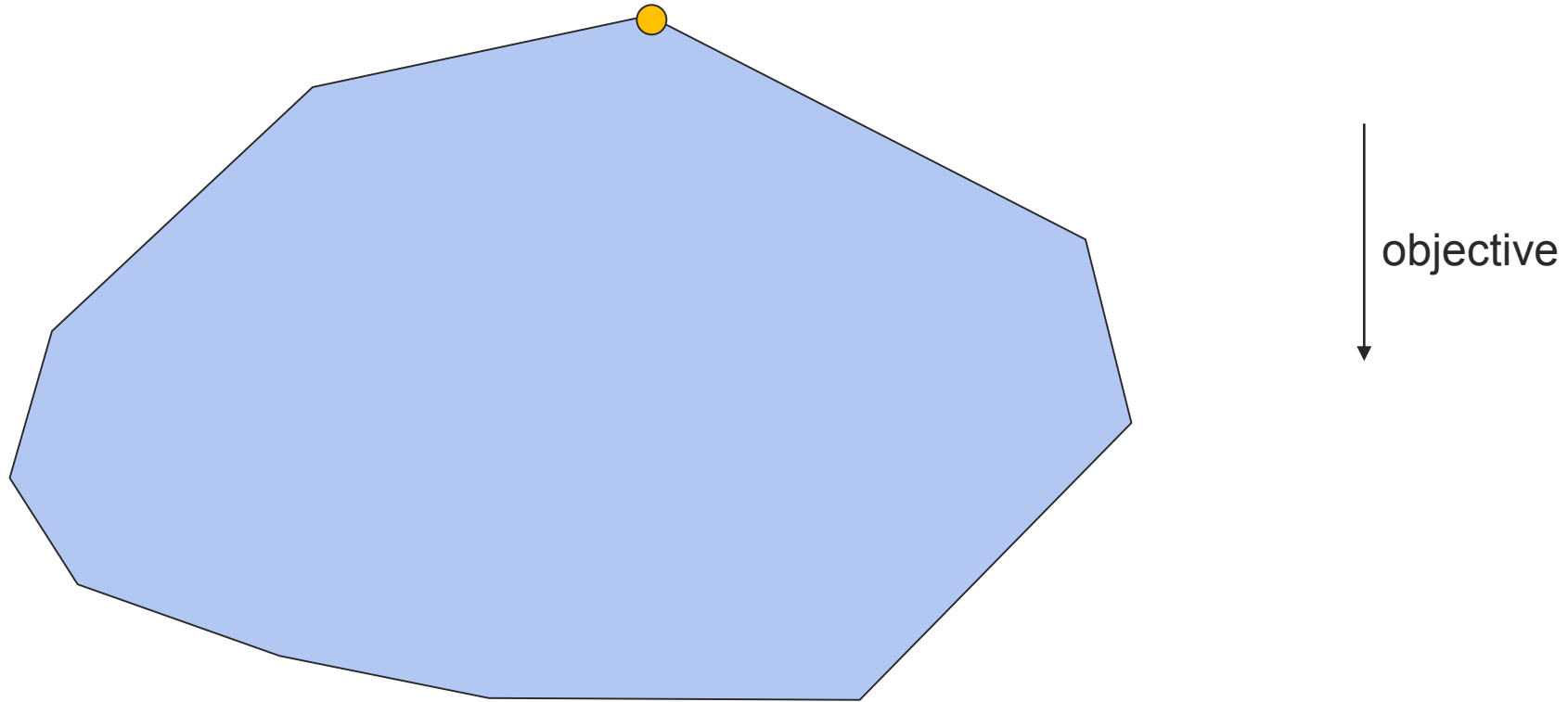
(if primal and dual are both feasible)

Karush-Kuhn-Tucker Conditions

- Conditions for LP optimality:
 - Primal feasibility: $Ax = b, x \geq 0$
 - Dual feasibility: $A^T y + z = c, z \geq 0$
 - Complementarity: $x^T z = 0$

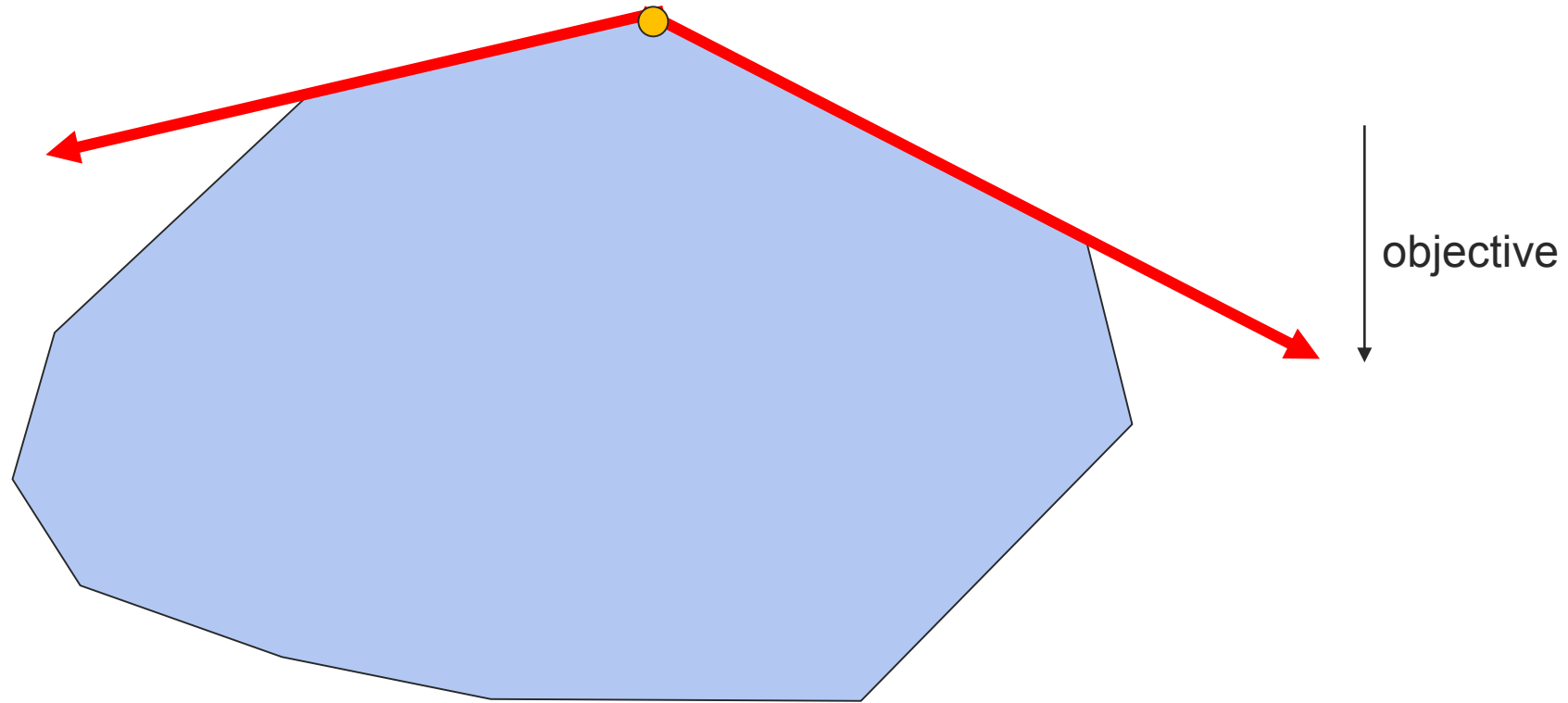
	Primal feasibility	Dual feasibility	Complementarity
Primal simplex	Maintain	Goal	Maintain
Dual simplex	Goal	Maintain	Maintain
Barrier	Goal	Goal	Goal

Simplex Algorithm



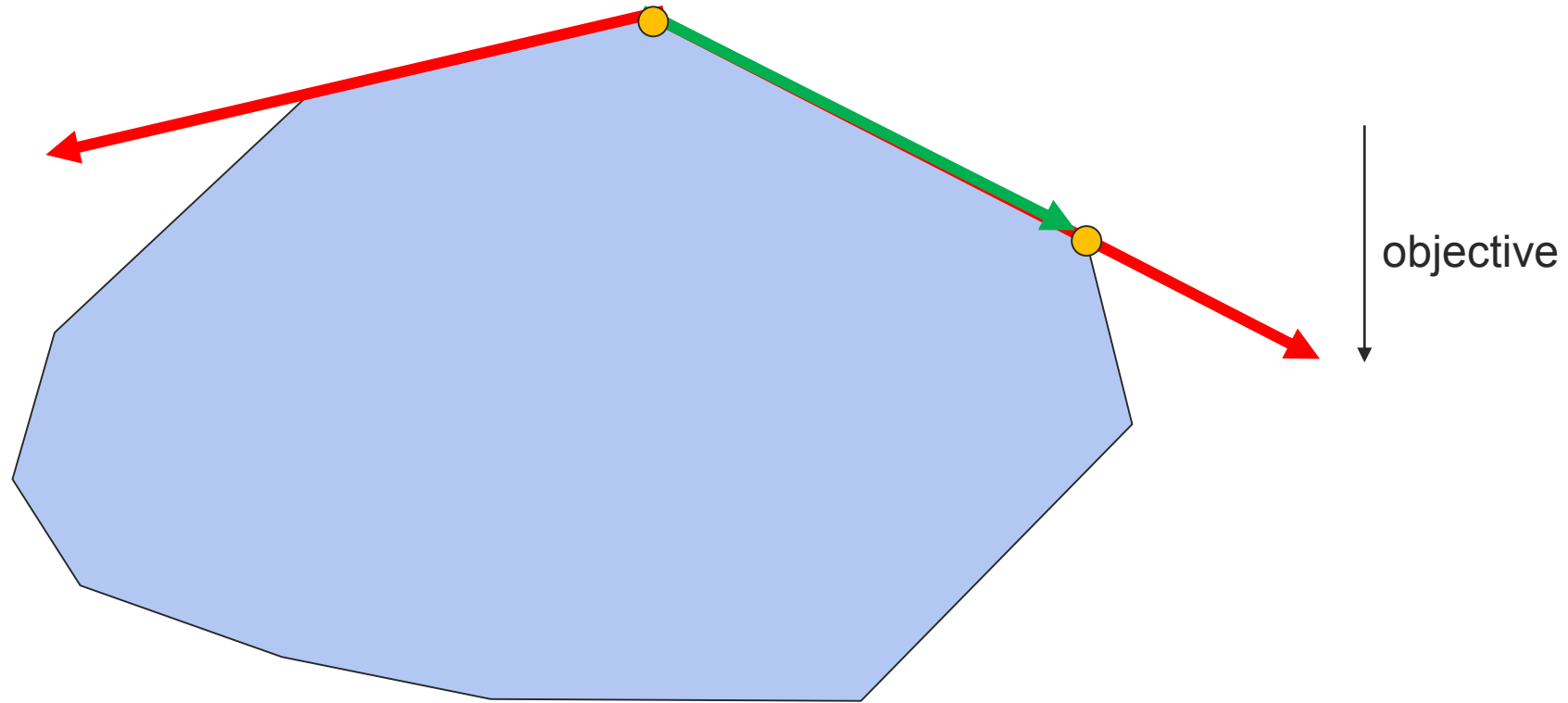
- **Phase 1:** find some feasible vertex solution

Simplex Algorithm



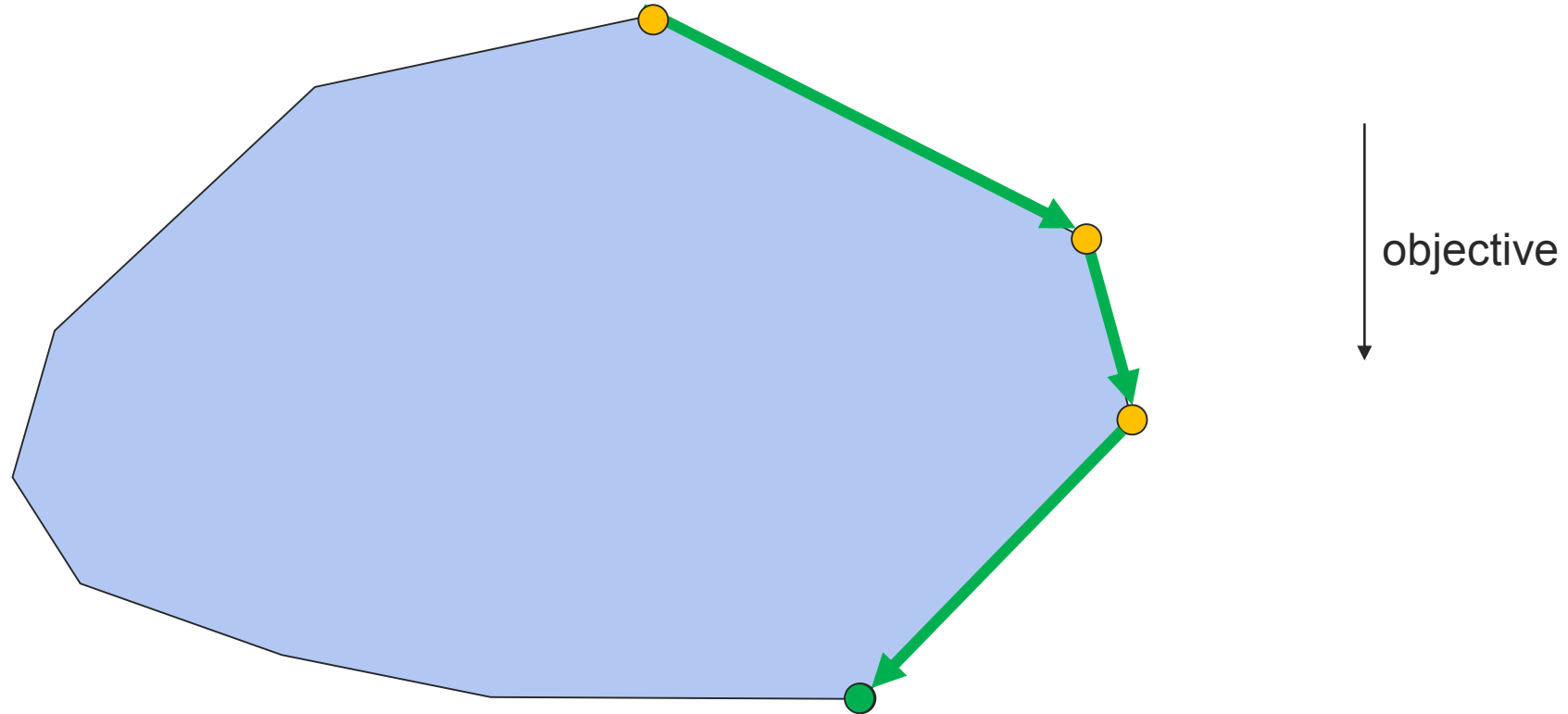
- **Pricing:** find directions in which objective improves and select one of them

Simplex Algorithm



- **Ratio test:** follow outgoing ray until next vertex is reached

Simplex Algorithm



- Iterate until no more improving direction is found
 - This is an optimal solution to the LP.

Simplex Algorithm – Linear Algebra

- Primal feasibility constraints

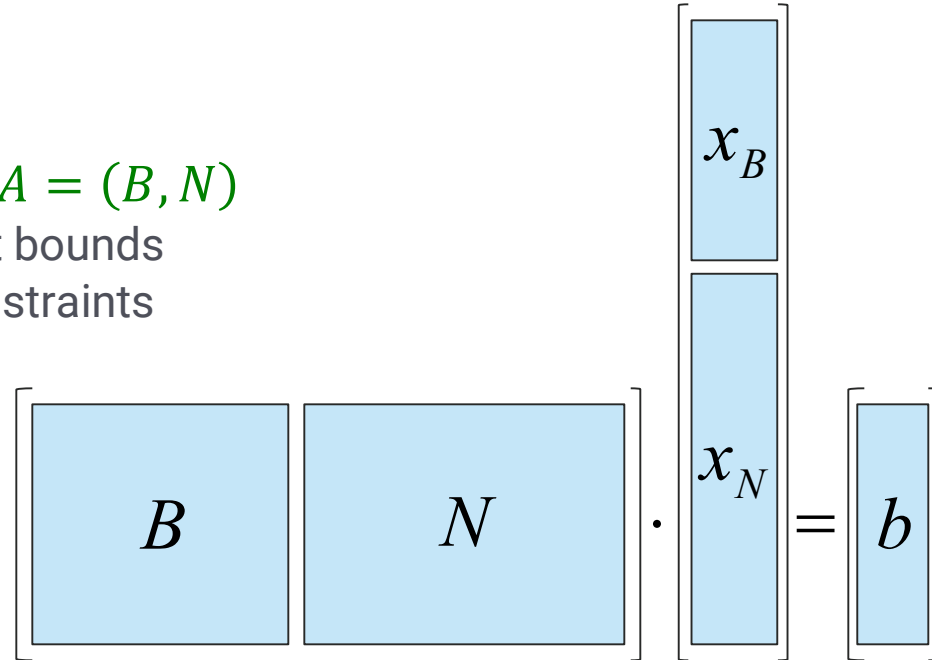
$$Ax = b$$

- Partition into basic and non-basic variables: $A = (B, N)$
 - Non-basic structural variables correspond to tight bounds
 - Non-basic slack variables correspond to tight constraints

$$Bx_B + Nx_N = b$$

- Solve for basic variables

$$x_B = B^{-1}(b - Nx_N)$$


$$\begin{bmatrix} B & N \end{bmatrix} \cdot \begin{bmatrix} x_B \\ x_N \end{bmatrix} = b$$

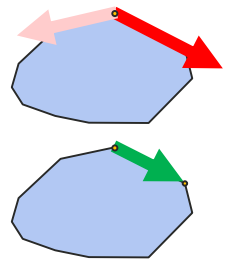
- Solved by maintaining

$$B = LU$$

Primal Simplex Algorithm – Pivoting

$$\begin{aligned} Bx_B &= b - Nx_N \\ B &= LU \\ LUx_B &= b - Nx_N \\ LW &= b - Nx_N \\ Ux_B &= w \end{aligned}$$

- **Simplex pivot:**
 - Choose a non-basic variable to enter the basis (Pricing)
 - Pick one with a negative reduced cost
 - Push one variable out of the basis (Ratio test)
 - Update primal and dual variables, reduced costs, basis, basis factors, etc.
 - Main work in each iteration: 2 (+1 for pricing norms) linear system solves
- **Apply simplex pivots until no more negative reduced cost variables exist (optimality)**



Simplex Algorithm – Pricing

- Dual variables

$$y = (B^{-1})^T c_B$$

- Reduced costs

$$z = c - A^T y$$

- Reduced costs give pricing information

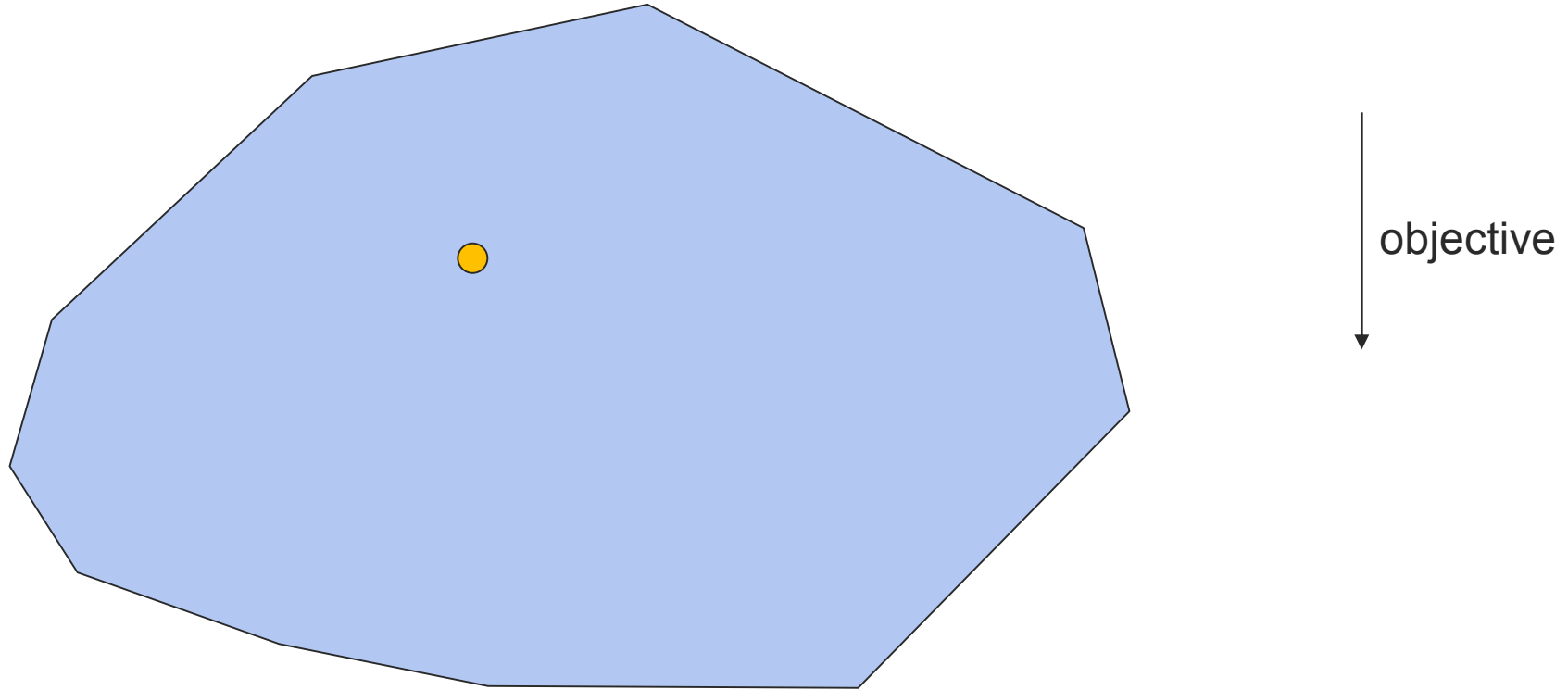
- Change in objective per unit change in variable value
- All reduced costs non-negative: proof of optimality
- Multiple variables with negative reduced costs: pick one of them
 - Steepest edge pricing: geometrically sound interpretation of what a "unit change" in the variable value means

Simplex Log

Iteration	Objective	Primal Inf.	Dual Inf.	Time
0	1.7748600e+04	6.627132e+03	0.000000e+00	0s
9643	1.1574611e+07	1.418653e+03	0.000000e+00	5s
14440	1.1607748e+07	4.793500e+00	0.000000e+00	10s
15213	1.1266396e+07	0.000000e+00	0.000000e+00	11s

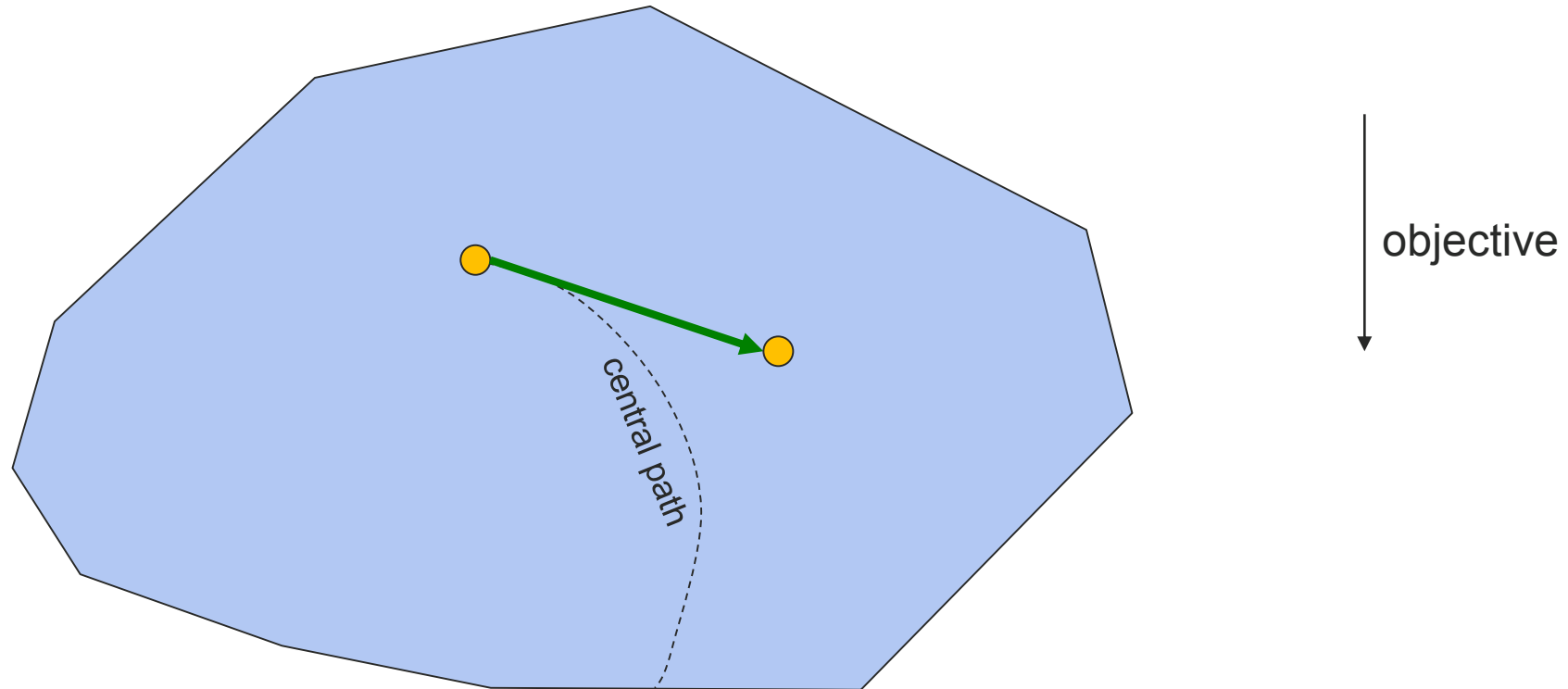
Solved in 15213 iterations and 10.86 seconds
Optimal objective 1.126639605e+07

Interior Point Method



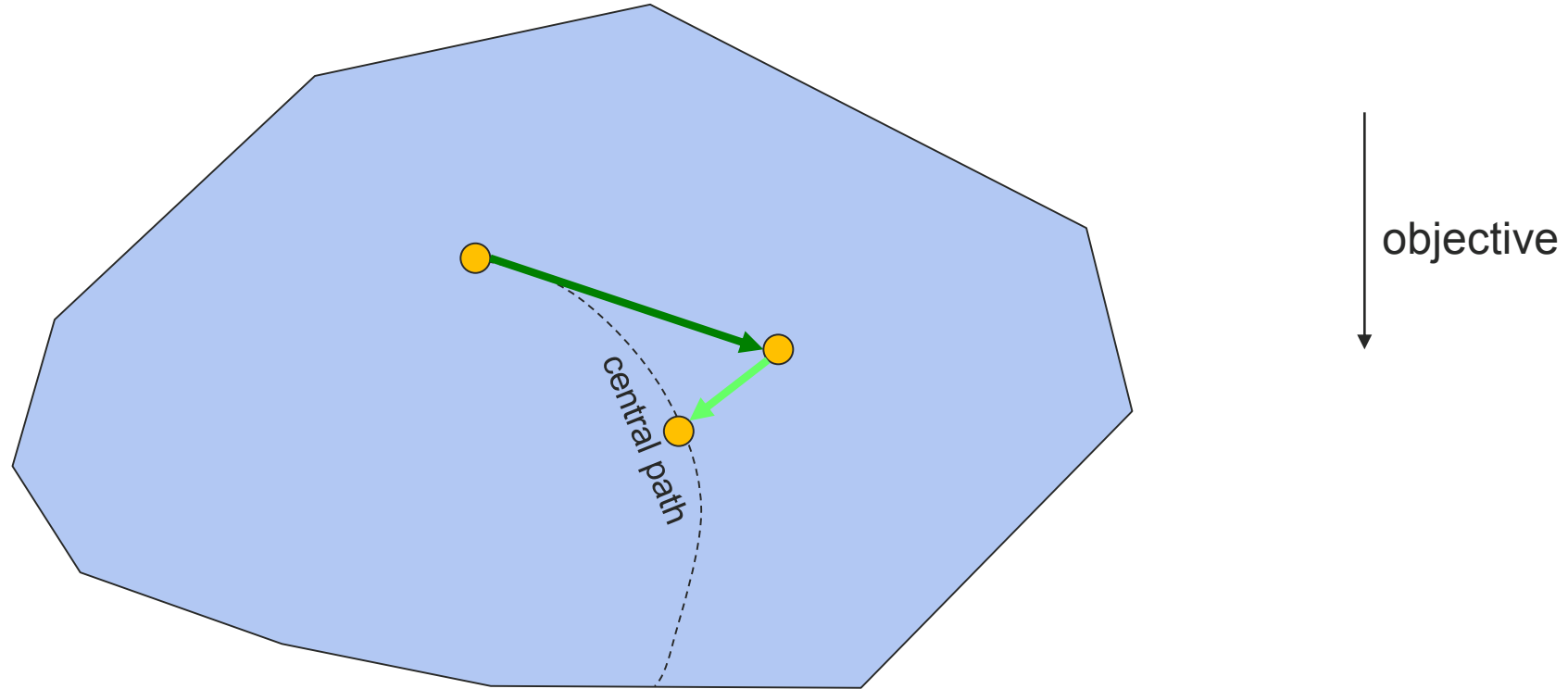
- Start with an interior point

Interior Point Method



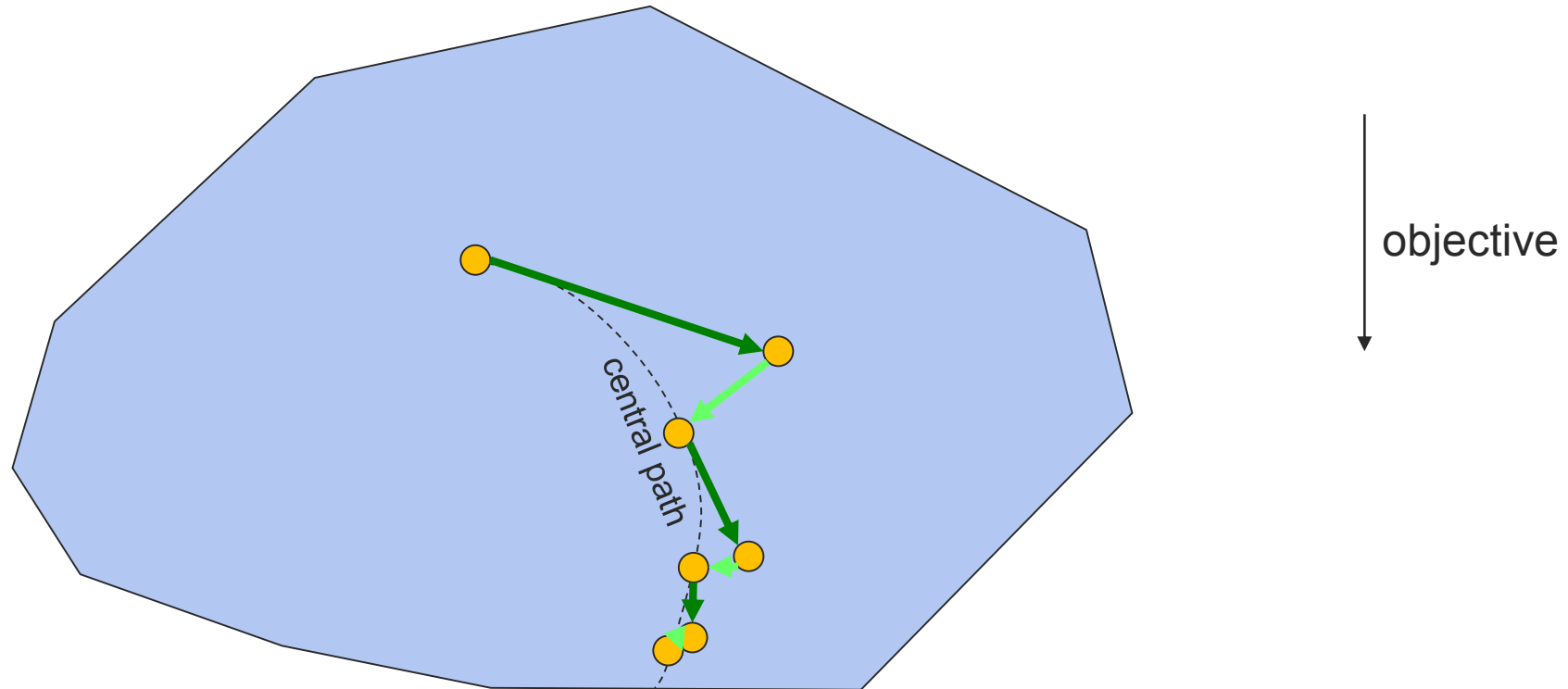
- **Predictor:** take step of certain length in tangent direction of central path

Interior Point Method



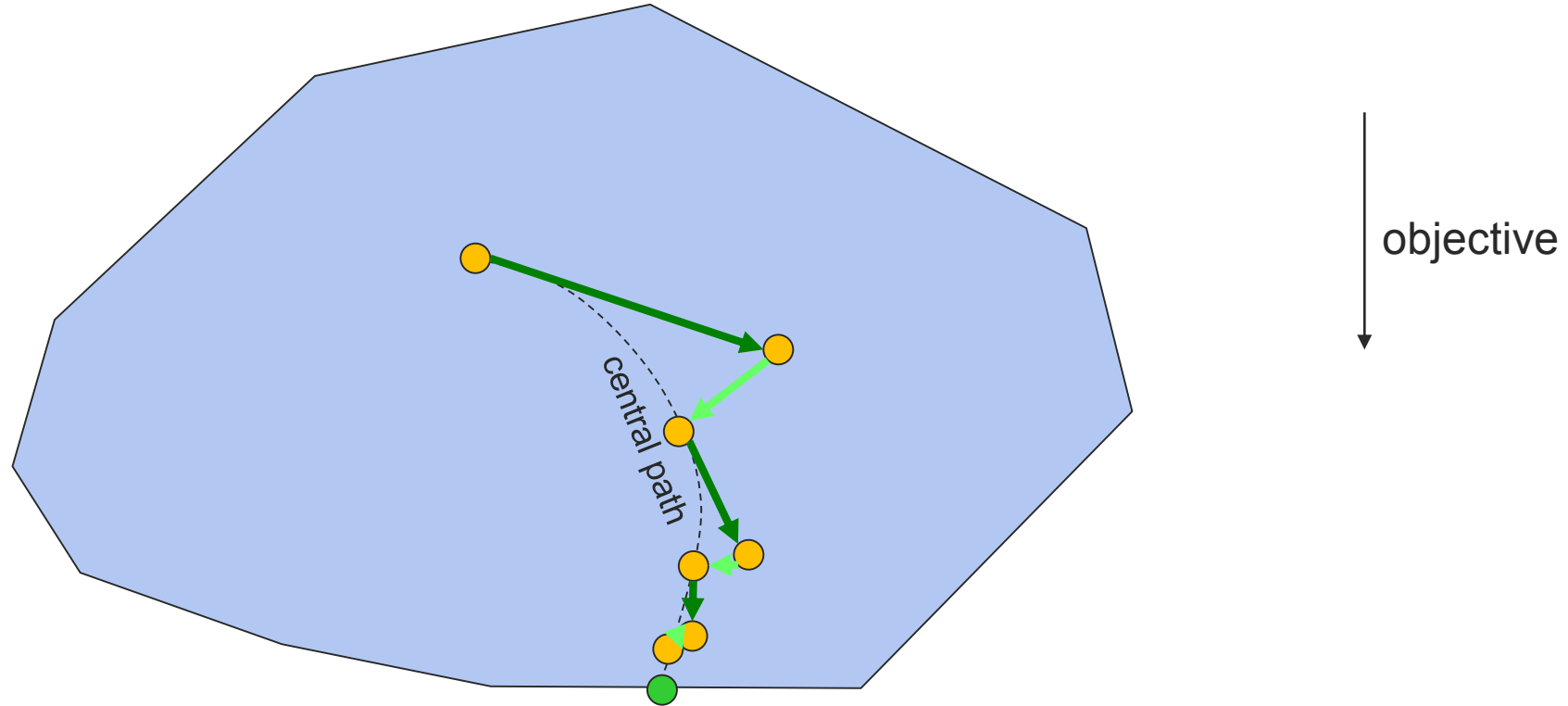
- **Corrector:** move back to central path

Interior Point Method



- Iterate until close enough to optimum

Interior Point Method



- Jump to the analytic center of the optimal face

Interior Point Method

- Basic algorithm (Karmarkar, Fiacco & McCormick, Dikin):

- Modify KKT conditions:

$$\begin{array}{rcl} Ax & = & b \\ A^T y + z & = & c \\ x^T z & = & 0 \end{array} \quad \longrightarrow \quad \begin{array}{rcl} Ax & = & b \\ A^T y + z & = & c \quad (X = \text{diag}(x)) \\ Xz & = & \mu e \end{array}$$

- Linearize complementarity condition:

$$\begin{pmatrix} -\theta & A^T \\ A & 0 \end{pmatrix} \begin{pmatrix} dx \\ dy \end{pmatrix} = \begin{pmatrix} r_2 \\ r_1 \end{pmatrix} \quad (\text{augmented system})$$

$$\theta_j = z_j / x_j, \quad x_j \cdot z_j = 0 \text{ at optimality, so } \theta_j \rightarrow 0 \text{ or } \infty$$

- Further simplification: $A\theta^{-1}A^T dy = b$ (normal equations)
- Iterate, reducing μ in each iteration
- Provable convergence

Computational Steps

- **Setup steps:**
 - Presolve (same for simplex)
 - Compute fill-reducing ordering
- **In each iteration:**
 - Form $A\theta^{-1}A^T$
 - Factor $A\theta^{-1}A^T = LDL^T$ (Cholesky factorization)
 - Solve $LDL^T x = b$
 - A few Ax and $A^T x$ computations
 - A bunch of vector operations
- **Post-processing steps:**
 - Perform crossover to a basic solution
 - Optional step, but usually required for LP relaxations in a MIP solve

Barrier Log

Barrier statistics:

AA' NZ : 2.836e+03
Factor NZ : 3.551e+03 (roughly 40 MBytes of memory)
Factor Ops : 1.739e+05 (less than 1 second per iteration)
Threads : 4

Iter	Objective		Residual		Compl	Time
	Primal	Dual	Primal	Dual		
0	1.30273209e+06	0.00000000e+00	5.90e+02	0.00e+00	7.32e+00	12s
1	1.04326180e+05	-5.84079103e+02	4.84e+01	1.69e+00	5.95e-01	12s
2	9.46325157e+03	-4.40392705e+02	2.92e+00	1.35e+00	5.46e-02	12s
3	3.66683689e+03	9.27381244e+02	1.94e-01	5.35e-01	1.41e-02	12s
4	3.37449982e+03	1.79938013e+03	1.29e-01	2.41e-01	7.64e-03	12s
5	3.13244138e+03	1.90266941e+03	8.89e-02	2.07e-01	6.00e-03	12s
6	2.71282610e+03	2.11401255e+03	3.20e-02	1.15e-01	2.96e-03	12s
7	2.48856811e+03	2.18107490e+03	1.06e-02	7.26e-02	1.56e-03	12s
8	2.35427593e+03	2.21183615e+03	3.20e-03	4.52e-02	7.36e-04	12s
9	2.30239737e+03	2.22464753e+03	1.53e-03	2.38e-02	4.03e-04	12s
10	2.25547118e+03	2.23096162e+03	3.00e-04	1.40e-02	1.30e-04	12s
11	2.24052450e+03	2.23917612e+03	4.10e-06	6.33e-04	7.20e-06	12s
12	2.23967243e+03	2.23966346e+03	2.01e-08	5.01e-06	4.82e-08	12s
13	2.23966667e+03	2.23966666e+03	1.11e-10	1.14e-13	4.81e-11	13s

Barrier solved model in 13 iterations and 12.51 seconds

Optimal objective 2.23966667e+03

Crossover Log

Barrier solved model in 13 iterations and 12.51 seconds
Optimal objective 2.2396667e+03

Root crossover log...

40 DPushes remaining with DInf 0.0000000e+00	13s
0 DPushes remaining with DInf 7.8159701e-14	13s
1176 PPushes remaining with PInf 0.0000000e+00	13s
0 PPushes remaining with PInf 0.0000000e+00	13s
Push phase complete: Pinf 0.0000000e+00, Dinf 1.2079227e-13	13s

Root simplex log...

Iteration	Objective	Primal Inf.	Dual Inf.	Time
1219	2.2396667e+03	0.0000000e+00	0.0000000e+00	13s
1219	2.2396667e+03	0.0000000e+00	0.0000000e+00	13s

Root relaxation: objective 2.239667e+03, 1219 iterations, 0.43 seconds

Essential Differences

- **Simplex:**
 - Thousand/millions of iterations on extremely sparse matrices
 - Each iteration extremely cheap
 - Few opportunities to exploit parallelism
 - Can be warm-started
- **Barrier:**
 - Dozens of expensive iterations
 - Much denser matrices
 - Lots of opportunities to exploit parallelism
 - Barrier warm-start still an open research topic

Concurrent LP

- **Run both simplex and barrier simultaneously**
 - Thread 1: dual simplex
 - Thread 2: barrier
 - Thread 3: barrier
 - Thread 4: barrier
 - Thread 5: primal simplex
 - Thread $n \geq 6$: barrier
- **Solution is reported by first one to finish**
- **Great use of multiple CPU cores**
- **Best mix of speed and robustness**
- **Deterministic and non-deterministic versions available**

LP Performance

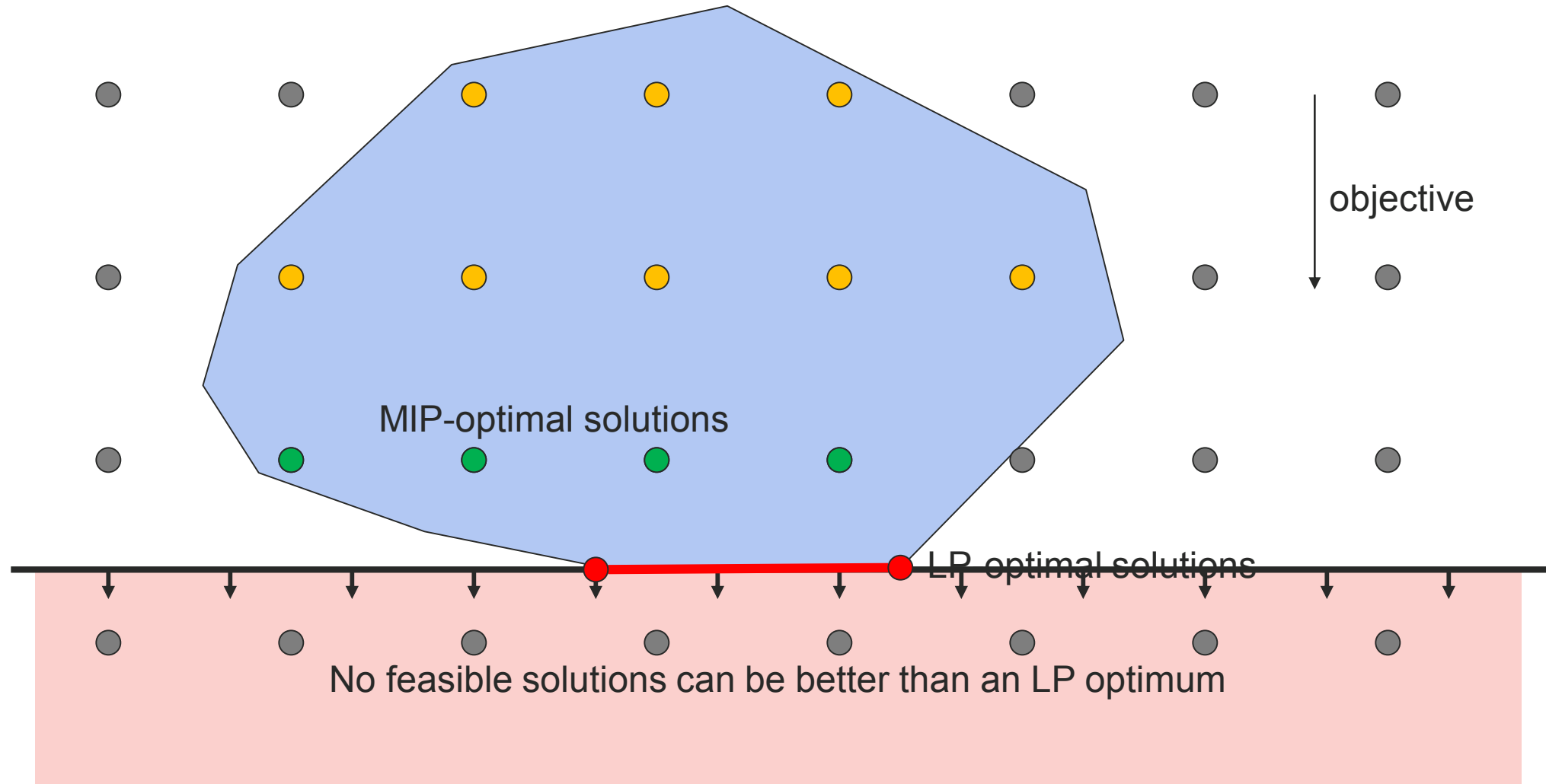
- **Performance results:**

- Gurobi 9.0, quad-core Xeon E3-1240
- Simplex on 1 core, barrier on 4 cores
- Concurrent with 1 thread dual, 3 threads barrier
- Models that take >1s

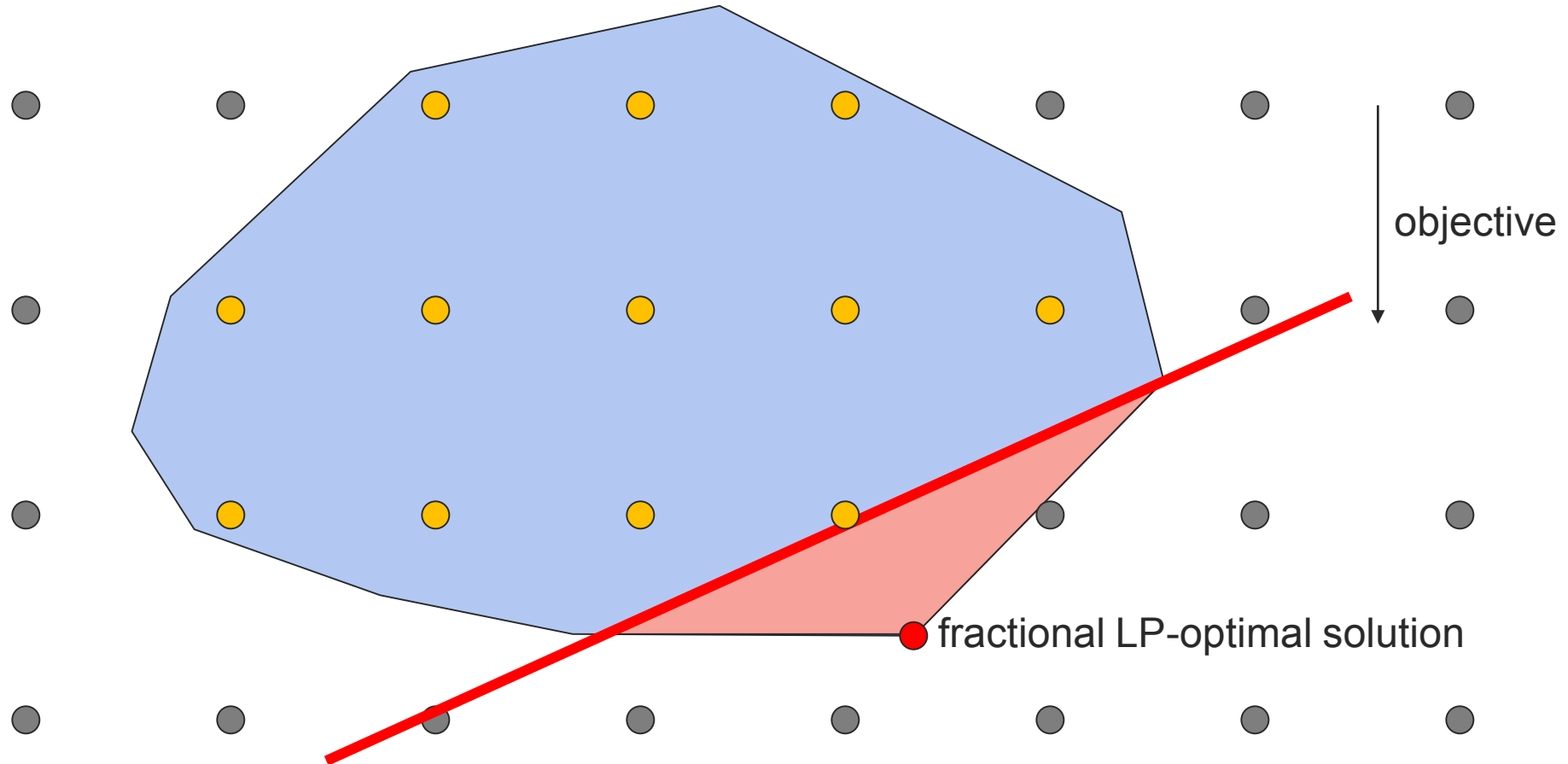
	GeoMean
Primal simplex	3.65
Dual simplex	2.25
Barrier	1.20
Concurrent	1.00
Deterministic Concurrent	1.13

MIP Algorithms

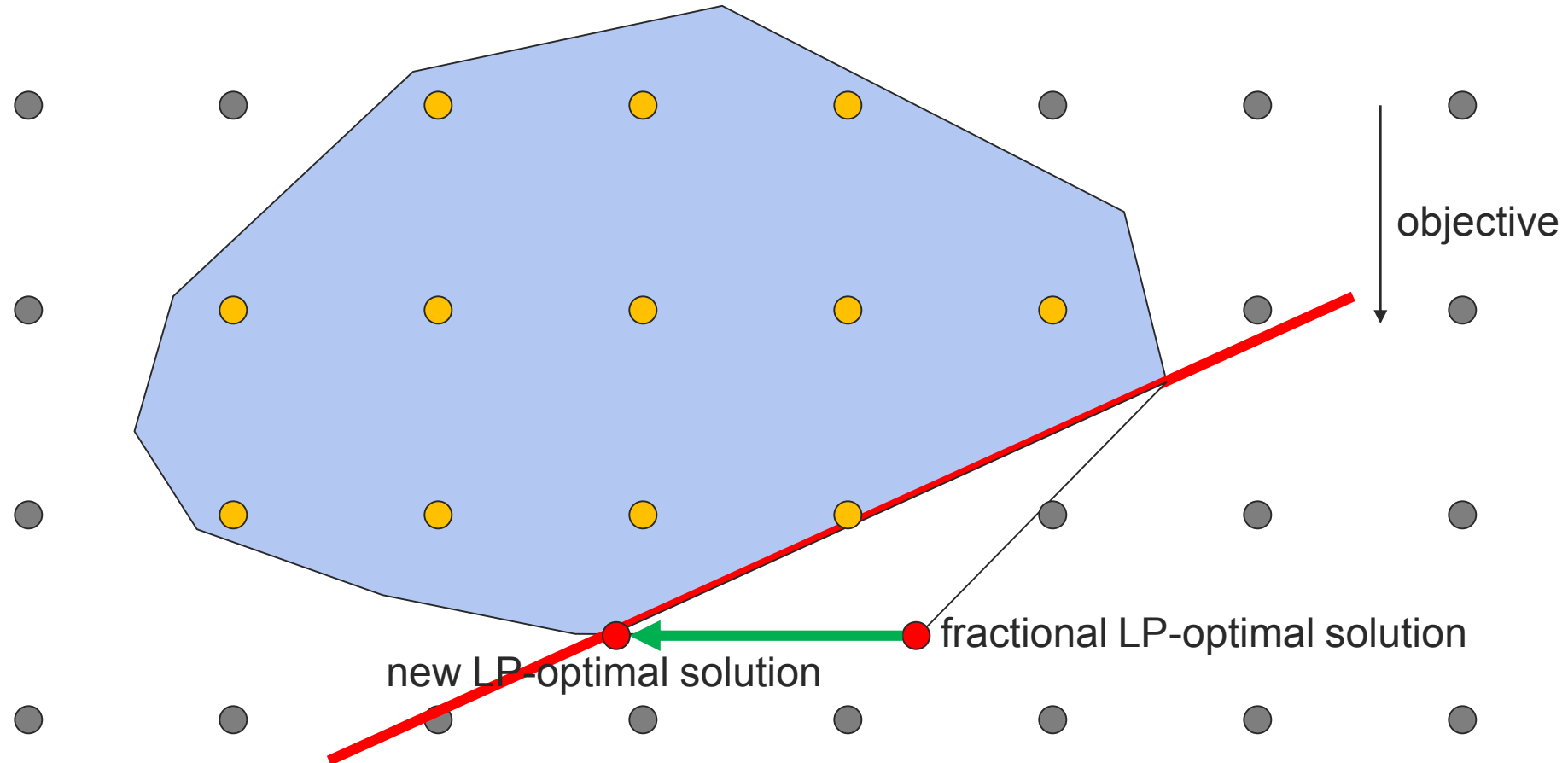
MIP – LP Relaxation



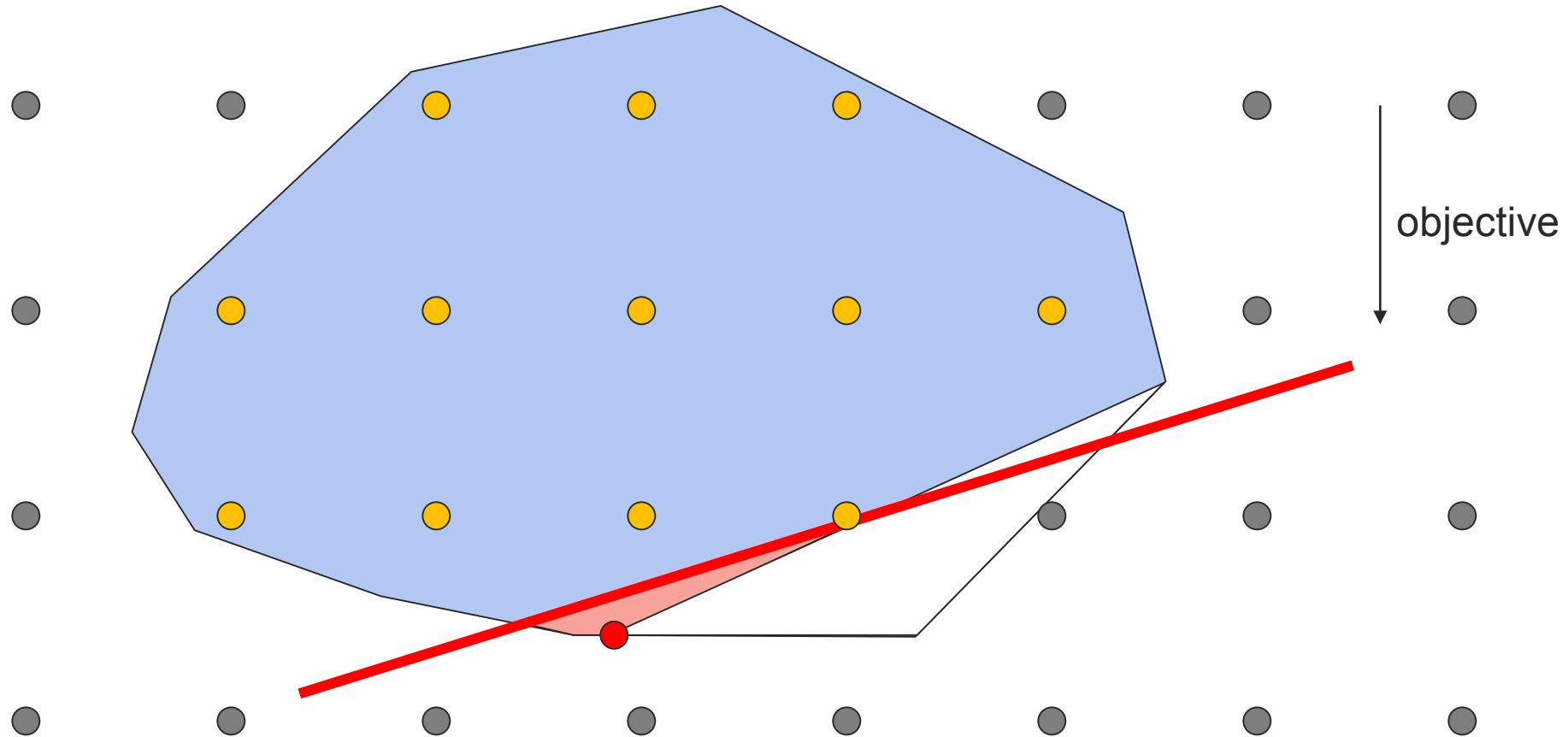
MIP – Cutting Planes



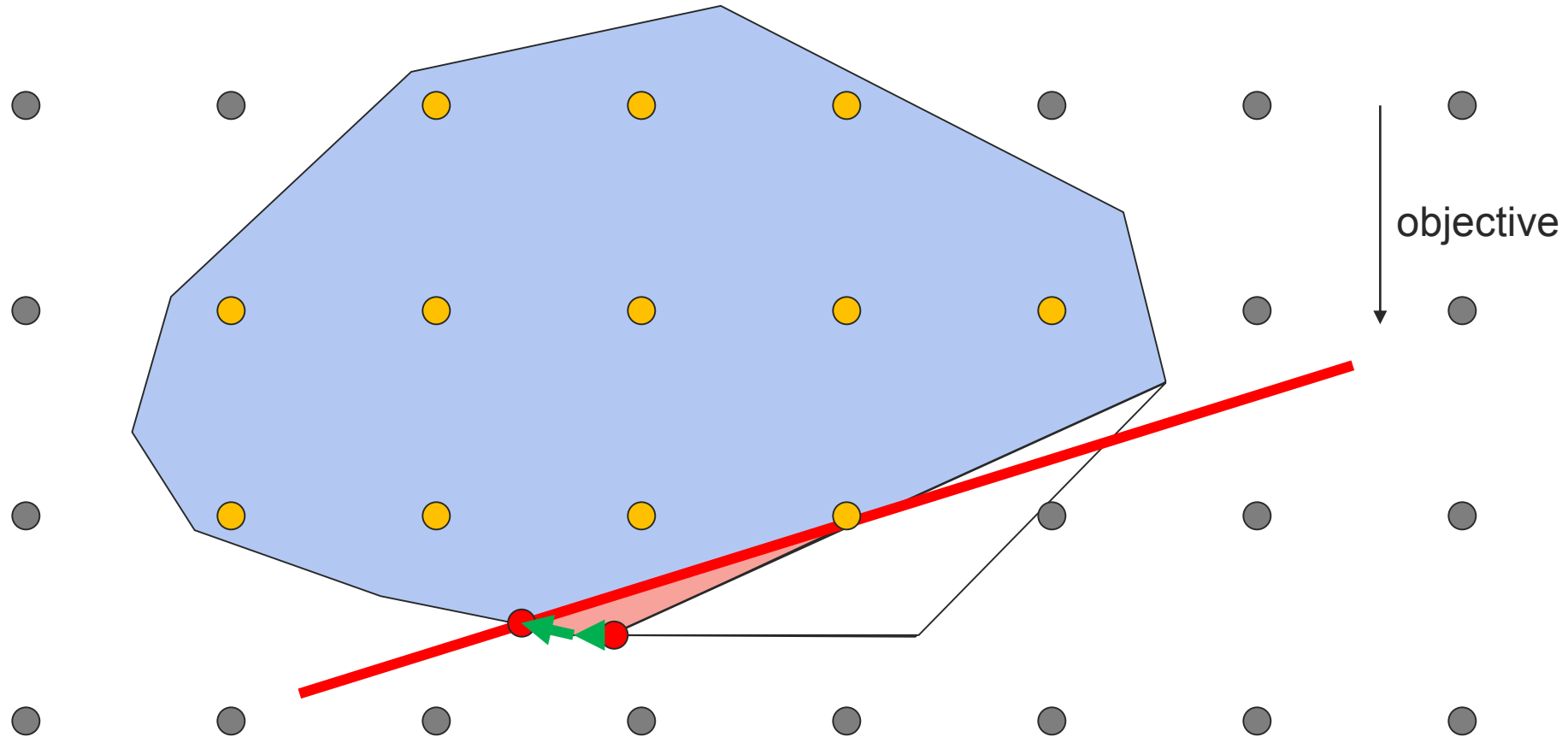
MIP – Cutting Planes



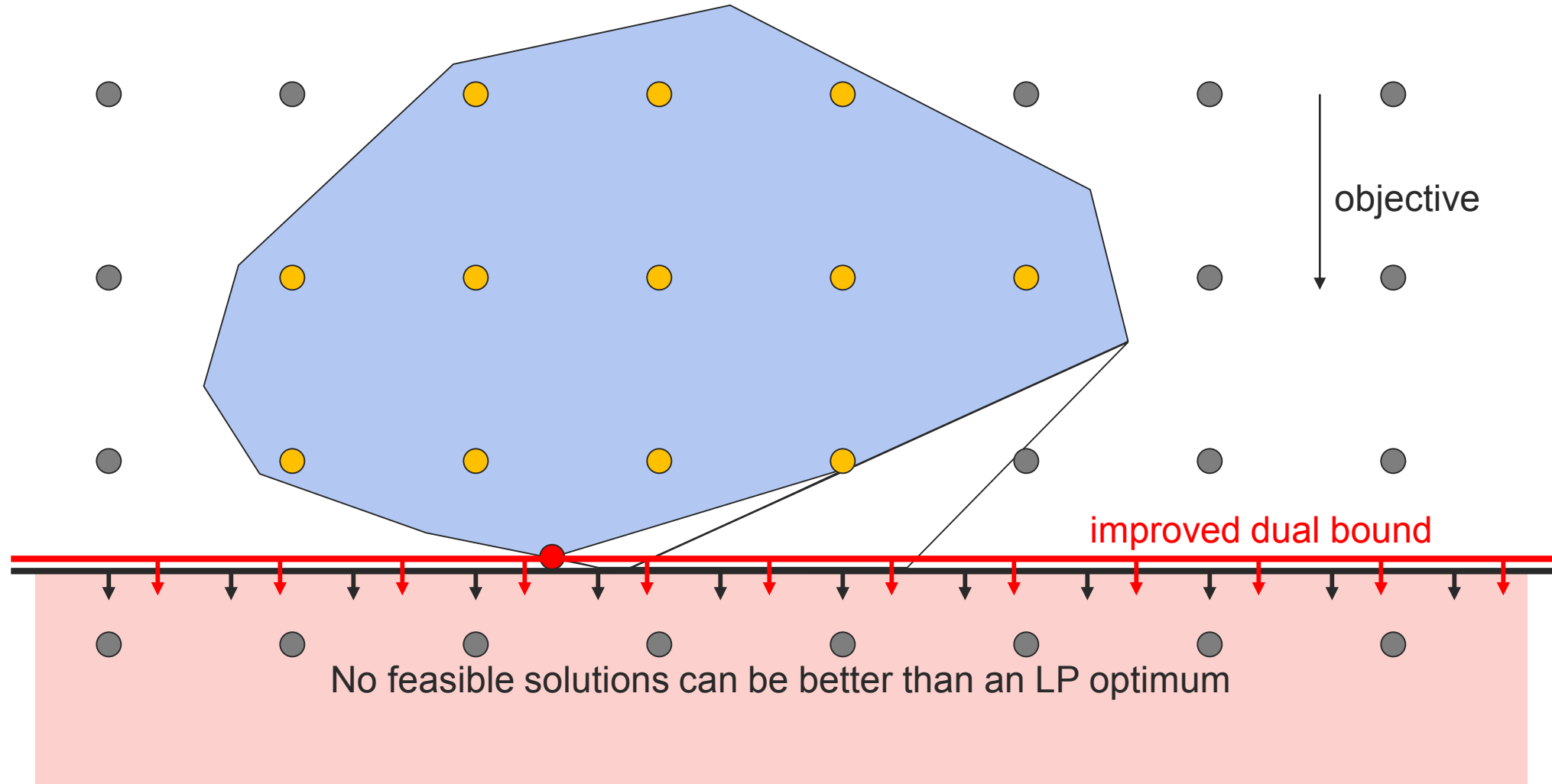
MIP – Cutting Planes



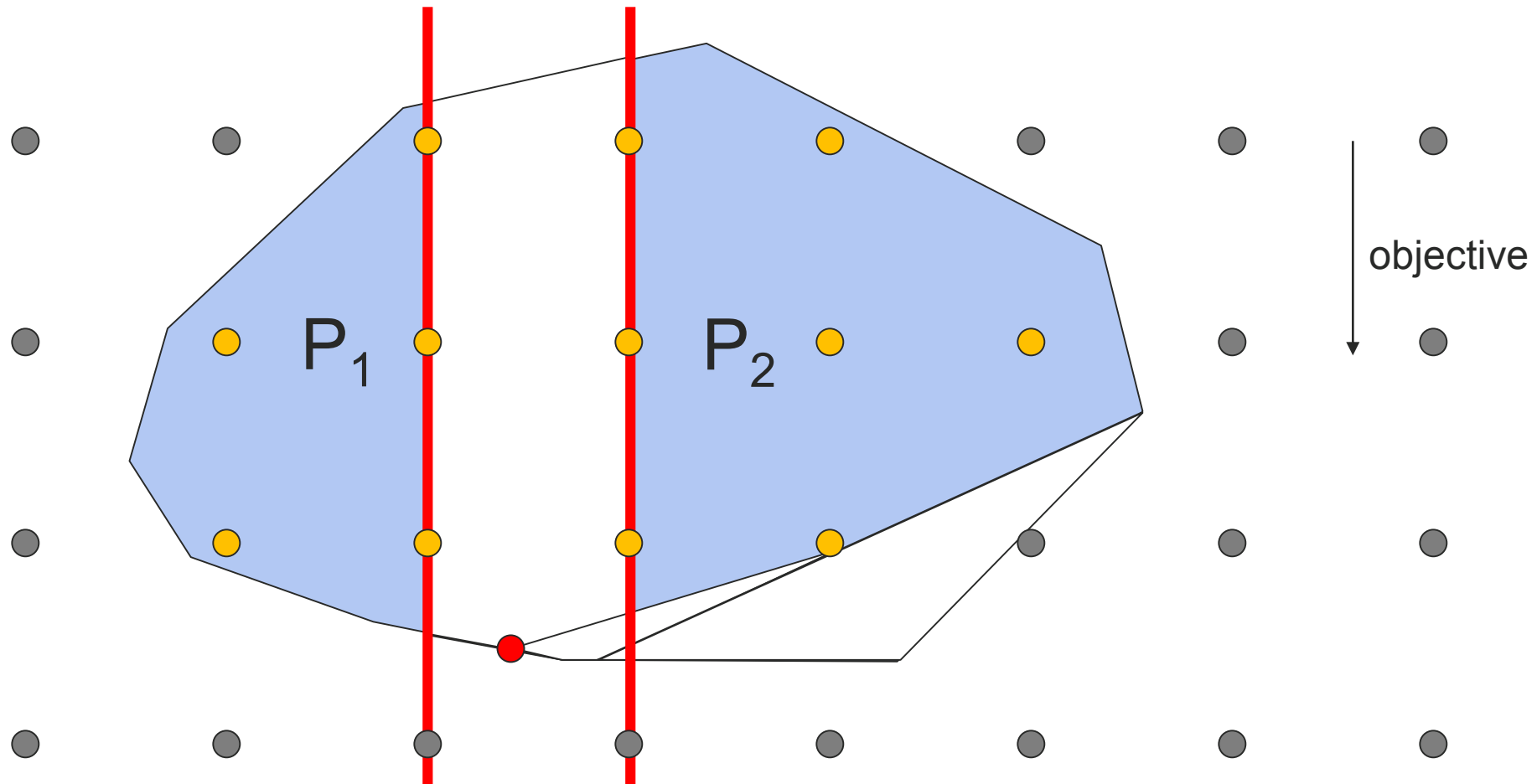
MIP – Cutting Planes



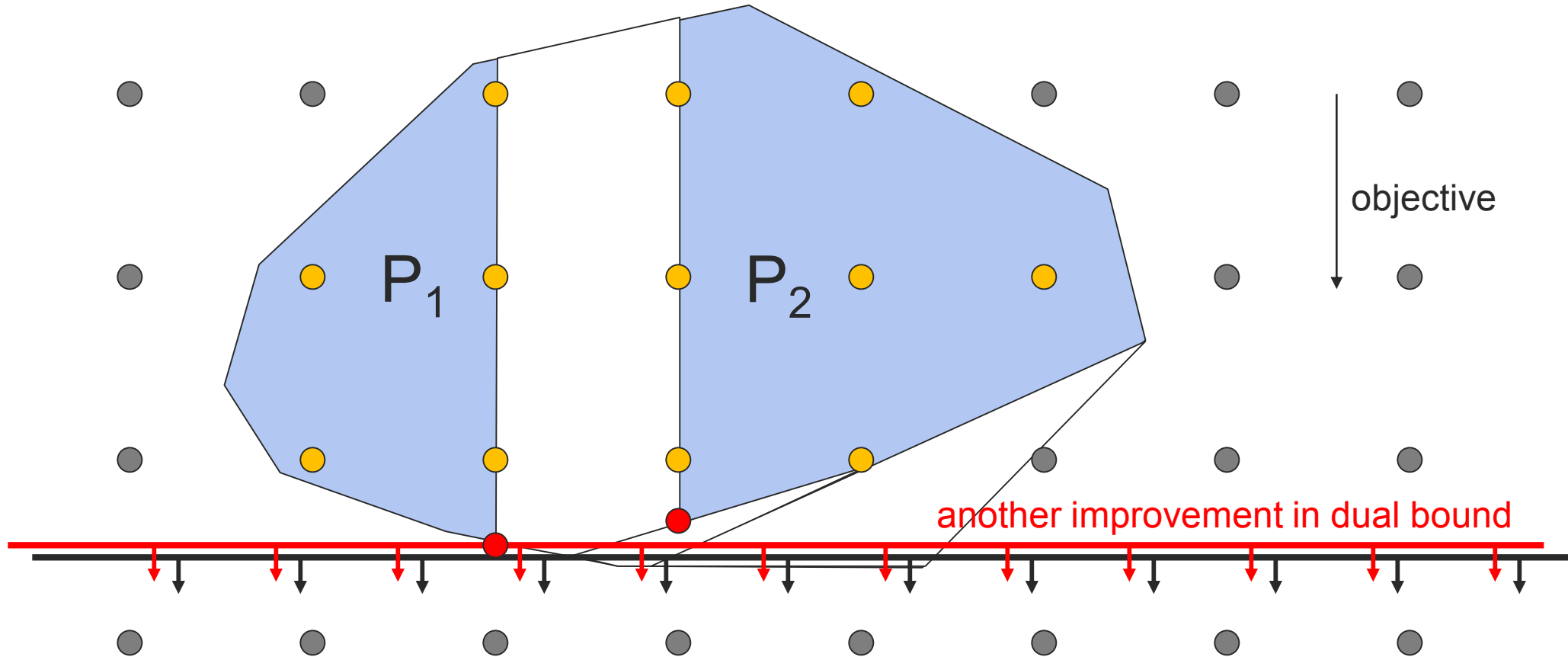
MIP – Cutting Planes



MIP – Branching

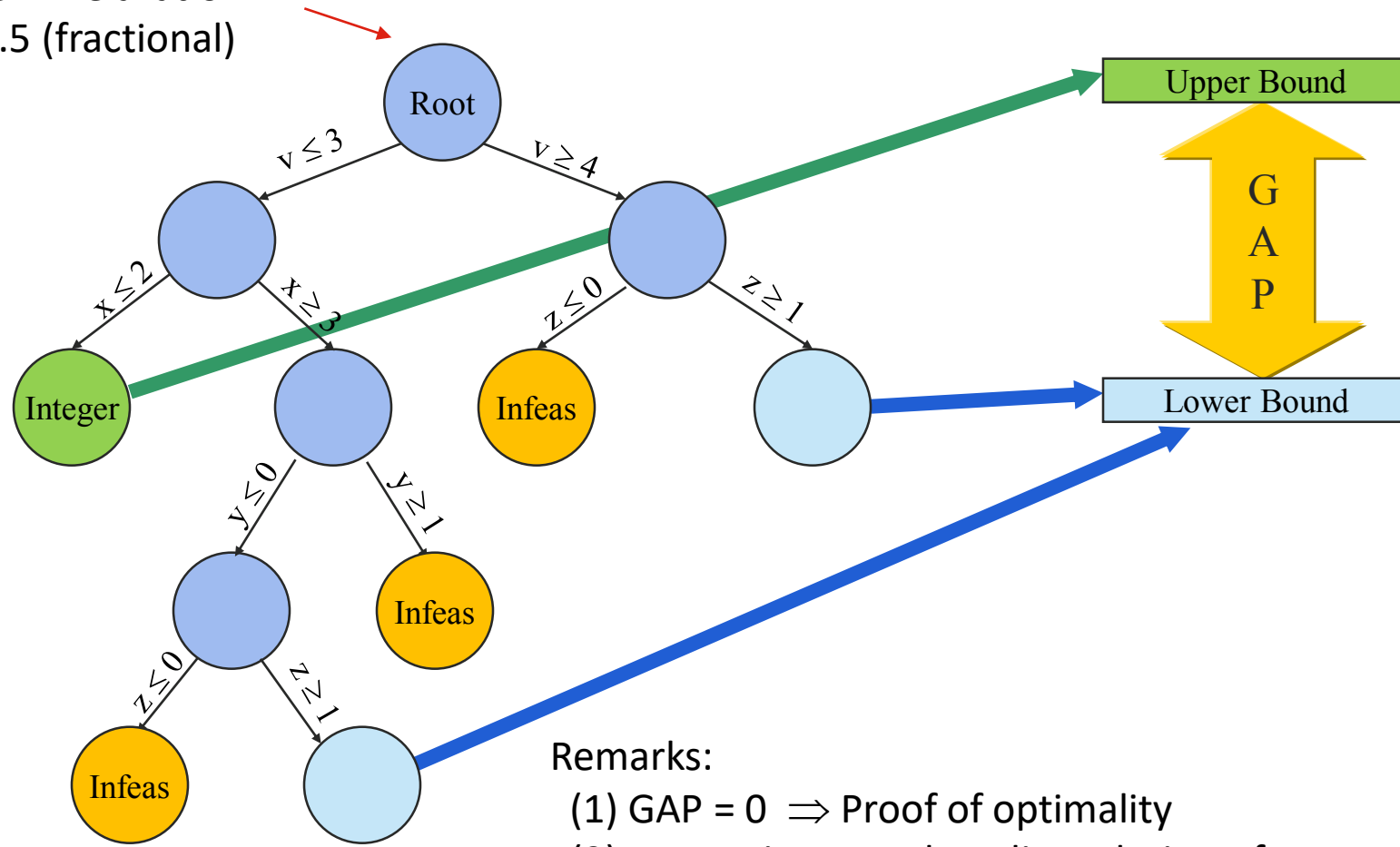


MIP – Branching



LP based Branch-and-Bound

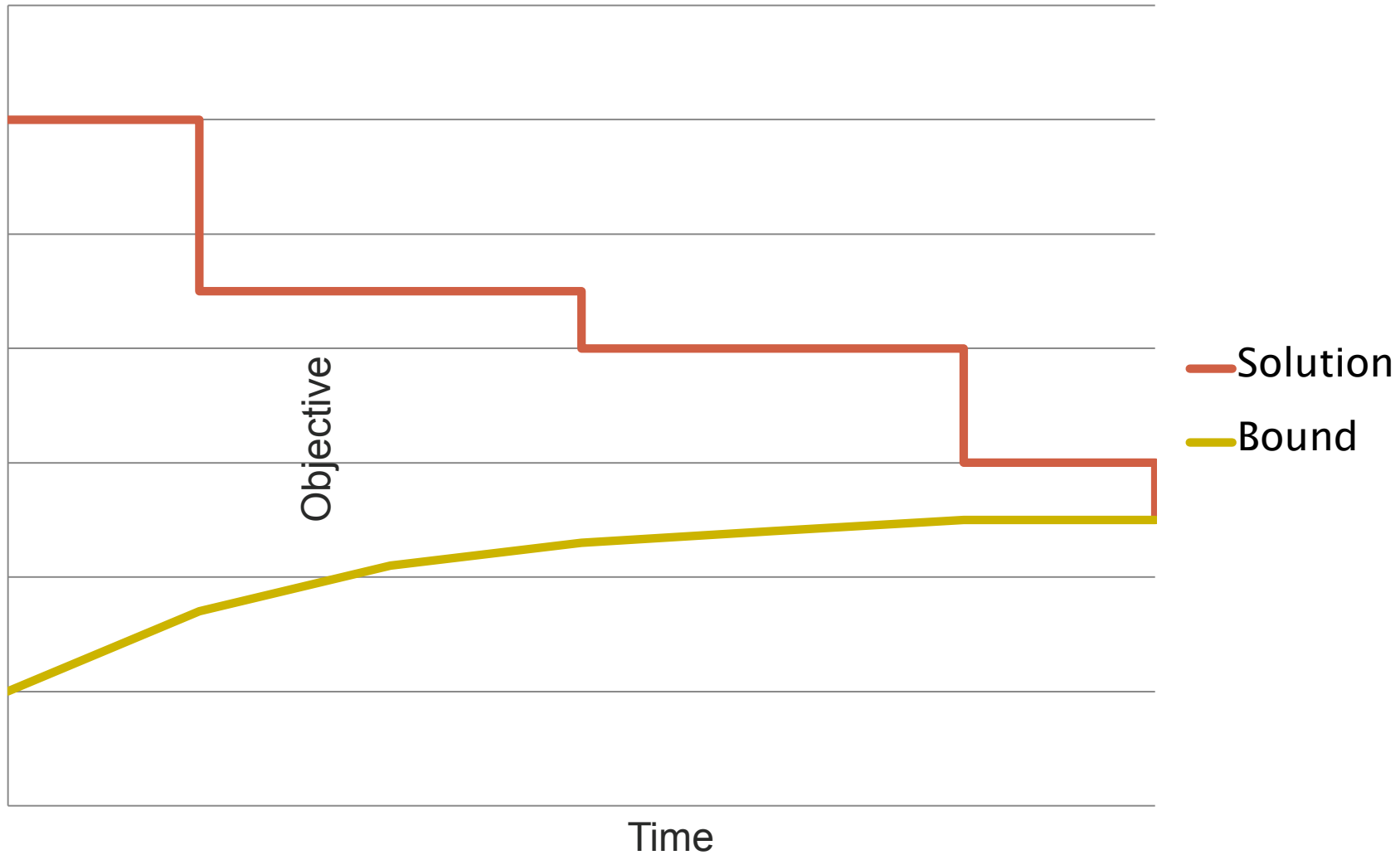
Solve LP relaxation:
 $v=3.5$ (fractional)



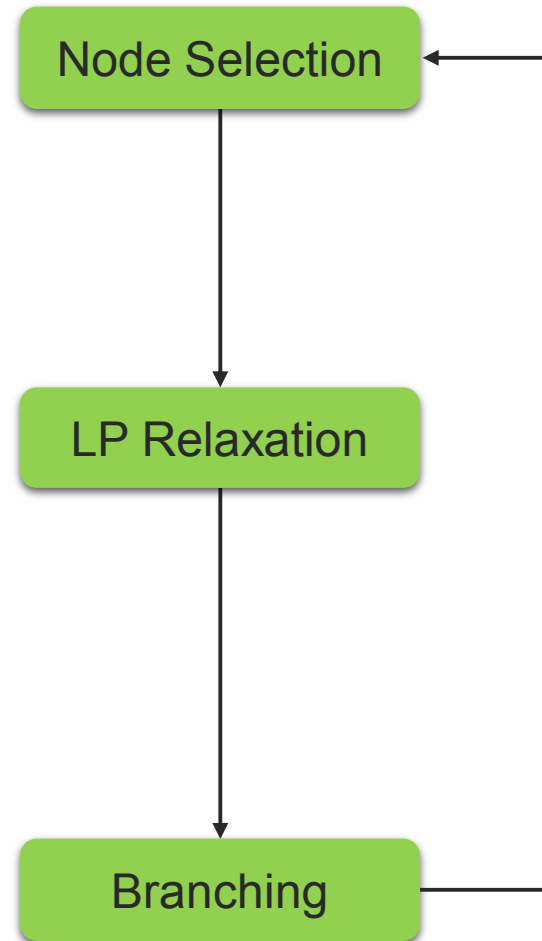
Remarks:

- (1) $\text{GAP} = 0 \Rightarrow$ Proof of optimality
- (2) In practice: good quality solution often enough

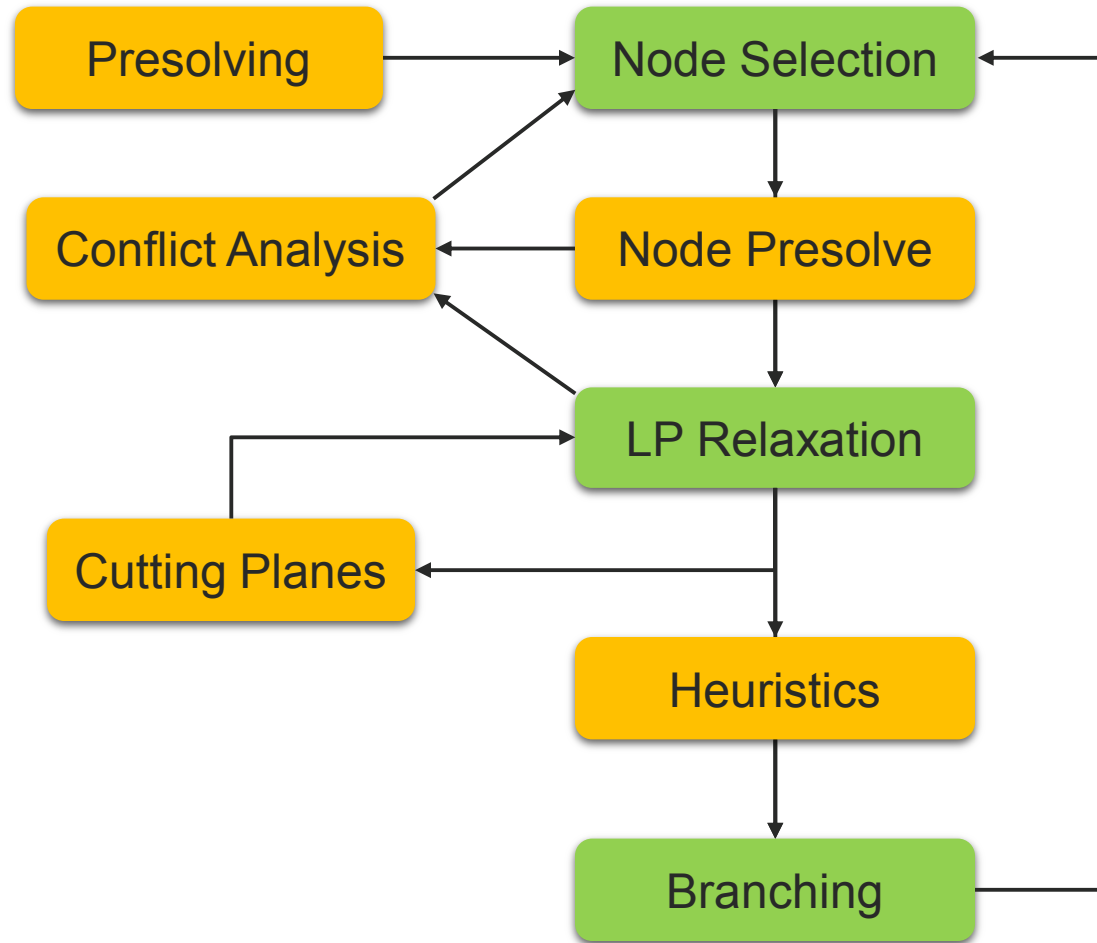
Solving a MIP Model



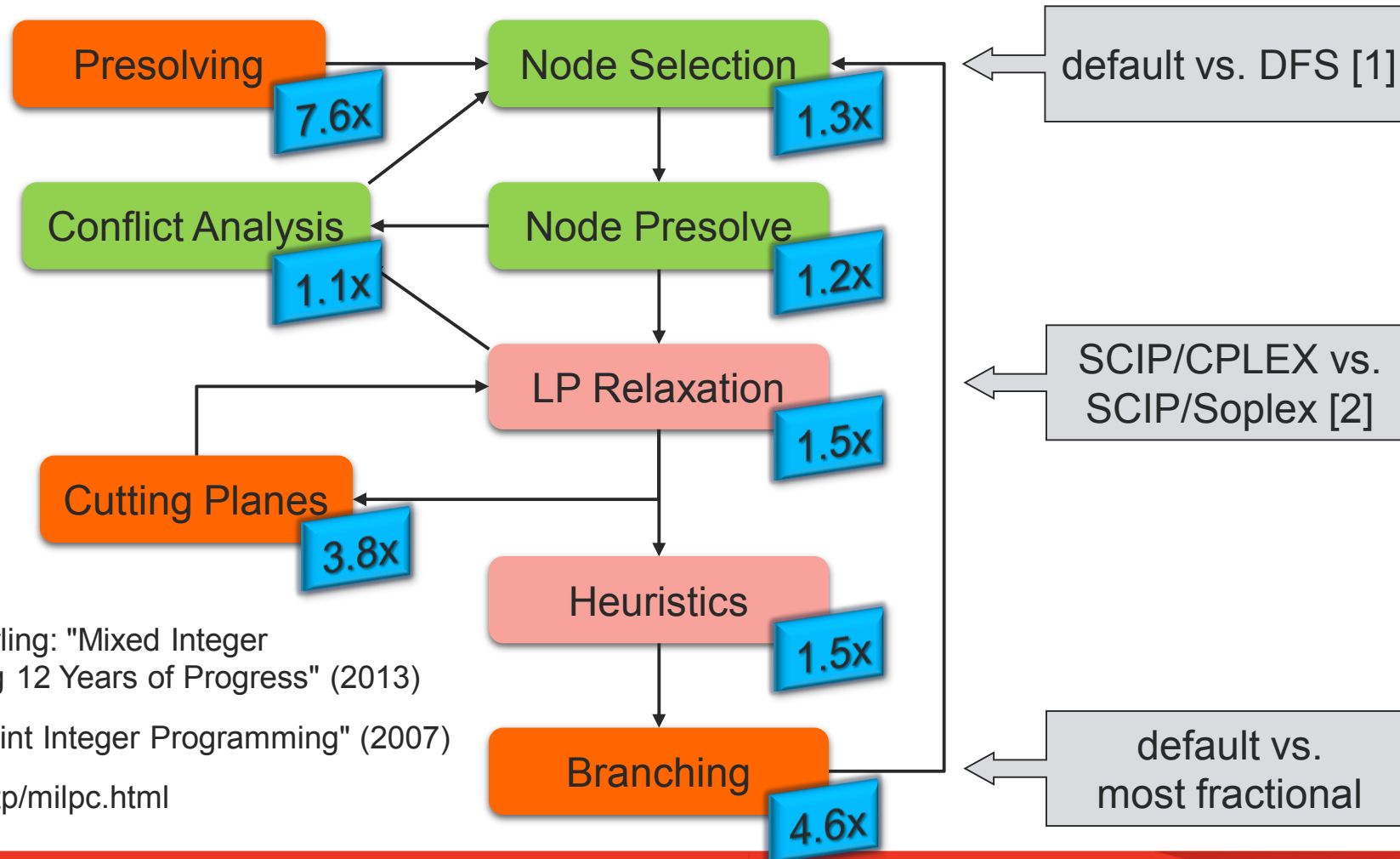
Branch-and-Bound



Branch-and-Cut



Performance Impact of MIP Solver Components (CPLEX 12.5 or SCIP)



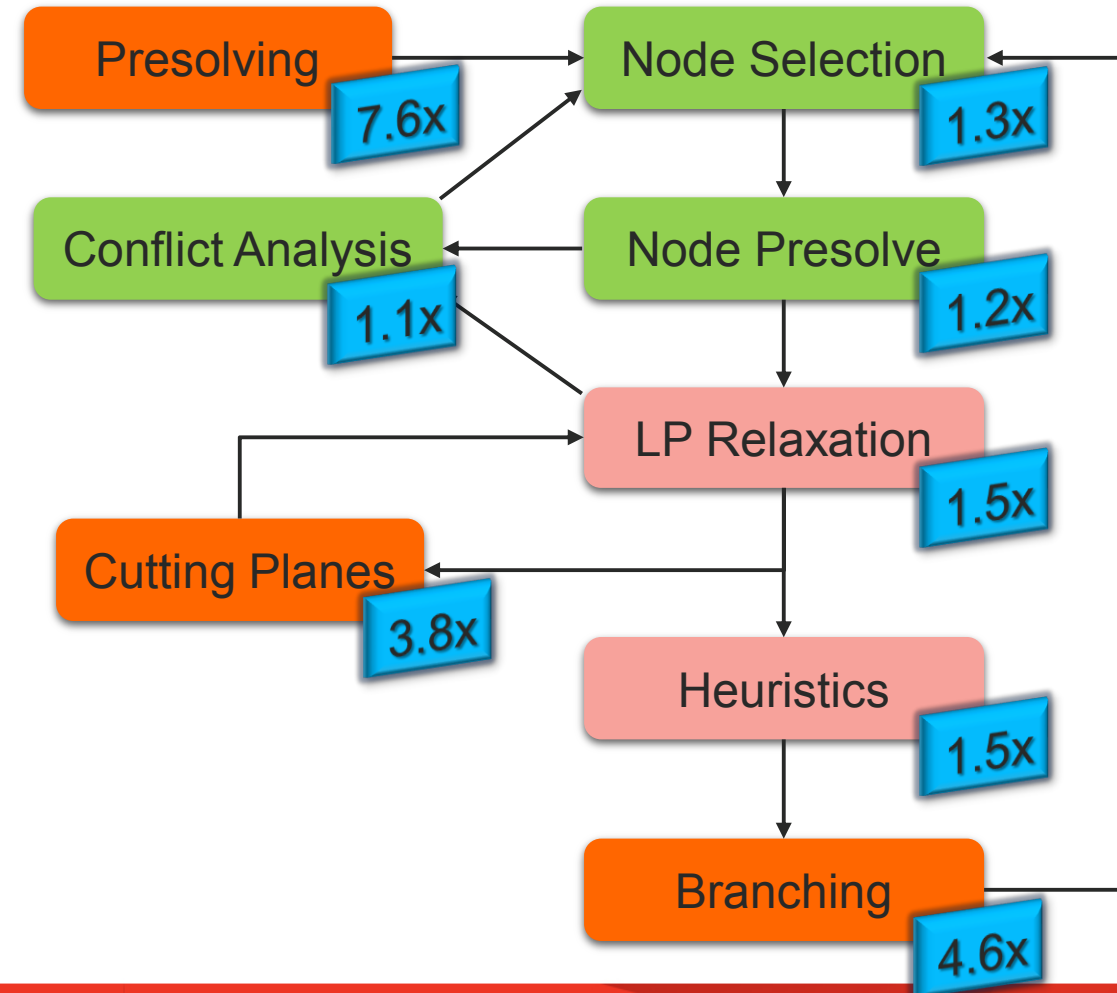
Achterberg and Wunderling: "Mixed Integer Programming: Analyzing 12 Years of Progress" (2013)

[1] Achterberg: "Constraint Integer Programming" (2007)

[2] <http://plato.asu.edu/ftp/milpc.html>

MIP Building Blocks

- **Presolve**
 - Tighten formulation and reduce problem size
- **Node selection**
 - Select next subproblem to process
- **Node presolve**
 - Additional presolve for subproblem
- **Solve continuous relaxations**
 - Ignoring integrality
 - Gives a bound on the optimal integral objective
- **Conflict analysis**
 - Learn from infeasible subproblems
- **Cutting planes**
 - Cut off relaxation solutions
- **Primal heuristics**
 - Find integer feasible solutions
- **Branching variable selection**
 - Crucial for limiting search tree size



Thank You!



GUROBI
OPTIMIZATION

The World's Fastest Solver